

IngresoQR

Multi-Tenant Gym Access Control System
Complete Technical Documentation



Version 2.0 | February 2026
FastAPI + React + MongoDB + Raspberry Pi

Table of Contents

- 1. System Overview
- 2. Architecture & Technology Stack
- 3. Project Structure (File Tree)
- 4. User Roles & Permissions
- 5. Database Schema (MongoDB Collections)
- 6. Backend API Reference
- 7. Frontend Pages & Routing
- 8. Authentication & Security
- 9. QR Code System (Dynamic & Static)
- 10. Raspberry Pi / IoT Integration
- 11. Payment System (Stripe & MercadoPago)
- 12. SaaS Multi-Tenant Model
- 13. Email System (SMTP per Gym)
- 14. Background Jobs (Cron Tasks)
- 15. Deployment Guide (Plesk VPS)
- 16. Environment Variables
- 17. Troubleshooting & FAQ

1. System Overview

IngresoQR is a comprehensive SaaS multi-tenant gym access control system. It provides gym owners with a complete solution for managing memberships, controlling physical access via QR codes, processing payments, tracking attendance, and managing classes. The system consists of three main components:

- Admin Dashboard: Web interface for gym administrators to manage all aspects of their gym
- Member PWA: Progressive Web App for gym members to access their QR code, book classes, view stats
- Raspberry Pi Controller: Physical IoT device that reads QR codes and controls entry/exit turnstiles via GPIO relays

Key Features

- Dynamic QR codes refreshing every 5/10/15 seconds (configurable per gym)
- Static QR mode for members with connectivity issues
- Anti-passback: Alternates entry/exit direction automatically
- Multi-tier roles: Super Admin, Gym Admin, Manager, Trainer, Member
- Stripe & MercadoPago payment integration (per-gym API keys)
- SaaS plan management with 12 configurable feature flags
- Class scheduling with recurring patterns and booking system
- POS (Point of Sale) for product inventory and sales
- Gamification with badges and streaks
- Custom routines assigned by trainers to members
- Custom registration forms per gym
- Guest pass system with QR access
- SMTP email system (configurable per gym)
- Analytics dashboard with hourly heatmaps and retention metrics
- Excel export of member data
- PDF sales reports
- Security: IP blocking after failed login attempts
- Member device management (limit devices per member)
- Kiosk mode for self-registration at the gym
- Dark/Light theme support

2. Architecture & Technology Stack

Backend

- Framework: FastAPI (Python 3.11+)
- Database: MongoDB (via Motor async driver)
- Authentication: JWT (python-jose + bcrypt)
- ASGI Server: Uvicorn
- File Storage: Local filesystem (/opt/gymaccess/uploads)
- Email: aiosmtplib (async SMTP)
- Payments: Stripe SDK, MercadoPago SDK
- Excel: openpyxl
- PDF: reportlab or fpdf2

Frontend

- Framework: React 18 (Create React App)
- Styling: Tailwind CSS + custom CSS variables for theming
- UI Components: Shadcn/UI (Radix-based)
- HTTP Client: Axios
- Routing: React Router v6
- Toasts: Sonner
- Fonts: Chivo (headings) + Manrope (body)
- Icons: Lucide React

IoT (Raspberry Pi)

- Hardware: Raspberry Pi 3B+ with 2-channel relay module
- QR Scanner: USB barcode/QR scanner (HID keyboard emulation)
- GPIO: BCM mode, Pin 12 (entry relay), Pin 16 (exit relay)
- Libraries: RPi.GPIO, evdev, requests, python-dotenv

Communication Flow

1. Member opens PWA -> Frontend requests QR from backend -> Backend generates HMAC-signed QR
2. Member shows QR at turnstile -> Raspberry Pi scanner reads QR
3. Raspberry Pi sends QR to POST /api/access/validate
4. Backend validates signature, checks membership, records access log
5. Backend responds valid/invalid -> Raspberry Pi activates entry or exit relay for 3 seconds

3. Project Structure

```

/app
|-- backend/
|   |-- server.py           # FastAPI app, routers, startup tasks
|   |-- models.py          # Pydantic request models
|   |-- auth.py            # JWT, password hashing, role checks
|   |-- database.py        # MongoDB connection (Motor)
|   |-- qr_utils.py        # QR generation & validation (HMAC)
|   |-- storage.py         # Local file storage helper
|   |-- requirements.txt    # Python dependencies
|   |-- requirements-prod.txt # Production dependencies
|   |-- .env               # Environment variables
|   |-- routes/
|       |-- auth_routes.py  # Login, register, profile
|       |-- gym_routes.py   # CRUD gyms, capacity, suspend
|       |-- member_routes.py # CRUD members, export Excel
|       |-- plan_routes.py  # Membership plans & assign
|       |-- access_routes.py # QR gen, validate, access logs
|       |-- device_routes.py # IoT device registration
|       |-- payment_routes.py # Stripe checkout, webhooks
|       |-- mercadopago_routes.py # MercadoPago integration
|       |-- class_routes.py  # Classes, schedules, bookings
|       |-- saas_routes.py   # SaaS plans, subscriptions
|       |-- pos_routes.py    # Products, sales, POS stats
|       |-- accounting_routes.py # Financial reports, PDF
|       |-- analytics_routes.py # Heatmaps, retention, stats
|       |-- notification_guest_routes.py # Notifications & guests
|       |-- security_routes.py # IP blocking, login attempts
|       |-- upload_routes.py  # Avatar, logo, product images
|       |-- form_routes.py    # Custom registration forms
|       |-- gamification_routes.py # Badges and streaks
|       |-- routine_routes.py # Trainer routines
|       |-- device_member_routes.py # Member device management
|       |-- device_management_routes.py # Device admin tools
|       |-- stripe_auto_routes.py # Automated Stripe billing
|       |-- misc_routes.py    # Email, dashboard, downloads
|       |-- demo_routes.py    # Demo data generation
|   |-- downloads/
|       |-- raspberry_access_control.py # Raspberry Pi script
|       |-- test_gpio.py               # GPIO test utility
|   |-- tests/                         # Pytest test files
|-- frontend/
|   |-- src/
|       |-- App.js           # Main router
|       |-- App.css          # Global styles + CSS variables
|       |-- context/AuthContext.js # Auth state management
|       |-- lib/api.js       # Axios API functions
|       |-- lib/utils.js     # Utility functions
|       |-- components/
|           |-- ErrorBoundary.js # React error catch
|           |-- MemberAvatar.js  # Avatar component
|           |-- ui/              # Shadcn UI components
|       |-- layouts/

```

```
| | | |-- AdminLayout.js      # Admin sidebar + header
| | | |-- PWALayout.js       # Member bottom nav
| | |-- pages/admin/         # 28 admin page components
| | |-- pages/pwa/           # 14 member page components
| |-- build/                  # Production build output
| |-- package.json
|-- memory/PRD.md             # Product requirements
```

4. User Roles & Permissions

Role Hierarchy

Role	gym_id	Description
super_admin	null	Full platform control. Manages all gyms, SaaS plans, devices, security.
gym_admin	UUID	Full control of their gym. Members, plans, payments, classes, settings.
gym_manager	UUID	Configurable permissions. Subset of gym_admin capabilities.
trainer	UUID	Manages classes assigned to them. Creates routines. Views own dashboard.
member	UUID	Accesses PWA. Views QR, books classes, manages profile.

Manager Permissions (Configurable)

Permission Key	Description
members_view	View member list
members_create	Create new members
members_edit	Edit member details
members_delete	Delete members
members_suspend	Suspend/Reactivate members
payments_register	Register manual payments
pos_sell	Process POS sales
pos_products	Manage POS products
access_view	View access logs
classes_manage	Manage classes & schedules
data_export	Export data (Excel)
notifications_send	Send notifications

5. Database Schema (MongoDB)

MongoDB is used as the primary database. All documents use a UUID string 'id' field as the primary key (not ObjectId). The '_id' field is always excluded from API responses.

Collection: admins

Admin users (super_admin, gym_admin, gym_manager, trainer)

```
id, email, password (bcrypt), name, role, gym_id, permissions[], active, created_at
```

Collection: gyms

Gym tenants

```
id, name, address, phone, email, logo_url, primary_color, qr_refresh_seconds, qr_mode, max_members, api_token, stripe_secret_key, stripe_currency, mercadopago_access_token, currency, smtp_host, smtp_port, smtp_user, smtp_password, smtp_from_email, email_templates[], saas_plan_id, status, created_at
```

Collection: members

Gym members

```
id, email, name, phone, code (6-char), gym_id, status (active/suspended/pending/blocked), gender, avatar_path, qr_mode, suspension_reason, can_bring_guests, max_guests_per_month, form_responses, created_at
```

Collection: plans

Membership plans

```
id, gym_id, name, description, price, duration_days, access_type, active, created_at
```

Collection: memberships

Active/expired memberships

```
id, member_id, plan_id, gym_id, start_date, end_date, status (active/expired), payment_id, created_at
```

Collection: access_logs

Entry/exit records

```
id, member_id, member_name, member_code, gym_id, direction (entrada/salida), timestamp, is_guest, guest_id
```

Collection: payment_transactions

Payment records (Stripe/Manual/MP)

```
id, session_id, member_id, plan_id, gym_id, amount, currency, status, payment_status, payment_method, notes, created_at
```

Collection: classes

Class definitions

```
id, gym_id, name, description, trainer_id, max_capacity, duration_minutes, class_type, recurring,
days_of_week[], start_time, end_time, start_date, end_date, active, created_at
```

Collection: class_schedules

Individual class occurrences

```
id, class_id, gym_id, date, start_time, end_time, trainer_id, max_capacity, current_bookings, status,
created_at
```

Collection: bookings

Class bookings by members

```
id, member_id, member_name, schedule_id, class_id, gym_id, date, status, checked_in, checked_in_at, created_at
```

Collection: devices

Registered IoT devices (Raspberry Pi)

```
id, gym_id, name, location, status, last_ping, created_at
```

Collection: notifications

Gym notifications for members

```
id, gym_id, title, message, notification_type, target, created_by, read_by[], created_at
```

Collection: guests

Guest passes

```
id, code, name, phone, invited_by, invited_by_name, gym_id, valid_until, valid_days, status, accesses,
created_at
```

Collection: saas_plans

Platform SaaS tier plans

```
id, name, max_members, has_qr_access, has_guest_passes, has_classes, has_pos, has_analytics, has_gamification,
has_routines, has_emailSMTP, has_stripe_members, has_mercadopago, has_iframes, has_advanced_accounting,
price_monthly, currency, active
```

Collection: pos_products

POS product inventory

```
id, gym_id, name, description, cost_price, sale_price, stock, category, barcode, image_path, active, created_at
```

Collection: pos_sales

POS sales transactions

```
id, gym_id, items[], total, currency, payment_method, member_id, registered_by, created_at
```

Collection: email_logs

Sent email history

```
id, gym_id, member_id, to_email, subject, email_type, status, error, sent_at
```

Collection: routines

Trainer-assigned routines

```
id, gym_id, name, description, trainer_id, member_id, days[], active, created_at
```

Collection: broadcasts

Platform-wide announcements (Super Admin)

```
id, title, message, priority, created_by, active, dismissed_by[], created_at
```

Collection: blocked_ips

Blocked IPs from brute force

```
ip, blocked_until, reason, blocked_at, last_identifier
```

Collection: login_attempts

Login attempt audit trail

```
ip, identifier, type, user_agent, timestamp, success
```

Collection: files

Uploaded file metadata

```
id, user_id, storage_path, original_filename, content_type, size, file_type, created_at
```

6. Backend API Reference

All API endpoints are prefixed with /api. Authentication is via Bearer JWT token in the Authorization header unless marked as public.

Authentication

Method	Endpoint	Description
POST	/api/auth/admin/register	Register a new admin user
POST	/api/auth/admin/login	Admin login, returns JWT token
POST	/api/auth/admin/impersonate/{gym_id}	Super Admin impersonates Gym Admin
PUT	/api/auth/admin/update-profile	Update email/password
POST	/api/auth/member/login?code=XXX	Member login by access code
GET	/api/auth/member/me	Get current member profile + gym + membership

Gyms

Method	Endpoint	Description
POST	/api/gyms	Create gym (+ optional gym admin)
GET	/api/gyms	List gyms (filtered by role)
GET	/api/gyms/{id}	Get gym details
PUT	/api/gyms/{id}	Update gym settings
DELETE	/api/gyms/{id}	Delete gym + all related data
PUT	/api/gyms/{id}/suspend	Toggle gym suspension
PUT	/api/gyms/{id}/payment-suspend	Toggle payment suspension
GET	/api/gyms/{id}/capacity	Get member capacity stats
GET	/api/gyms/{id}/public-info	Public: gym name, logo, payment status

Members

Method	Endpoint	Description
POST	/api/members	Create member (admin)
POST	/api/members/register	Public self-registration
GET	/api/members	List members (?gym_id, ?status)
GET	/api/members/{id}	Get member details
PUT	/api/members/{id}	Update member
POST	/api/members/{id}/suspend	Suspend with reason

DELETE	/api/members/{id}	Delete member + related data
POST	/api/members/check-expired-memberships	Suspend expired members
GET	/api/members/export/excel	Export members to .xlsx

Plans & Memberships

Method	Endpoint	Description
POST	/api/plans	Create membership plan
GET	/api/plans	List plans
GET	/api/plans/public/{gym_id}	Public: plans for a gym
PUT	/api/plans/{id}	Update plan
DELETE	/api/plans/{id}	Soft-delete plan
POST	/api/memberships	Assign membership to member
GET	/api/memberships	List memberships
GET	/api/memberships/expiring?days=10	Expiring memberships

Access & QR

Method	Endpoint	Description
GET	/api/qr/generate	Generate QR for logged-in member
POST	/api/access/validate	PUBLIC: Validate QR (Raspberry Pi)
GET	/api/access/logs	Access logs (?gym_id, ?member_id, ?dates)
GET	/api/access/stats	Access statistics
GET	/api/access/stats/daily?days=7	Daily access breakdown
GET	/api/access/stats/hourly	Hourly distribution
GET	/api/access/stats/member/{id}	Individual member stats
PUT	/api/members/{id}/qr-mode	Set static/dynamic QR

Payments

Method	Endpoint	Description
POST	/api/payments/checkout?plan_id=X	Create Stripe checkout session
GET	/api/payments/status/{session_id}	Check payment status
POST	/api/payments/manual	Register manual payment
GET	/api/payments/history	Payment history
GET	/api/gyms/{id}/stripe-config	Get Stripe config (masked)
PUT	/api/gyms/{id}/stripe-config	Update Stripe API key

POST	/api/webhook/stripe	Stripe webhook handler
------	---------------------	------------------------

Classes & Bookings

Method	Endpoint	Description
POST	/api/classes	Create class
GET	/api/classes	List classes
PUT	/api/classes/{id}	Update class (regenerates schedules)
DELETE	/api/classes/{id}	Delete class + schedules
POST	/api/schedules	Create individual schedule
GET	/api/schedules	List schedules
GET	/api/schedules/public/{gym_id}	Public: upcoming schedules
POST	/api/bookings	Book a class (member)
GET	/api/bookings/member	Member's bookings
DELETE	/api/bookings/{id}	Cancel booking
POST	/api/bookings/{id}/checkin	Check-in for class
POST	/api/classes/cleanup-stale-schedules	Remove orphaned schedules

SaaS Management

Method	Endpoint	Description
POST	/api/saas/plans	Create SaaS plan (Super Admin)
GET	/api/saas/plans	List SaaS plans
PUT	/api/saas/plans/{id}	Update SaaS plan
PUT	/api/gyms/{id}/saas-plan	Assign SaaS plan to gym
GET	/api/gyms/{id}/saas-features	Get gym's active features
GET	/api/saas/my-subscription	Gym Admin: my subscription
POST	/api/saas/subscribe	Subscribe to SaaS plan (Stripe)

POS

Method	Endpoint	Description
POST	/api/pos/products	Create product
GET	/api/pos/products	List products
PUT	/api/pos/products/{id}	Update product
POST	/api/pos/sales	Create sale
GET	/api/pos/sales	List sales

GET	/api/pos/stats	POS statistics
-----	----------------	----------------

Security

Method	Endpoint	Description
GET	/api/security/blocked-ips	List blocked IPs
DELETE	/api/security/blocked-ips/{ip}	Unblock IP
GET	/api/security/login-attempts	Login attempt audit
GET	/api/security/stats	Security statistics

7. Frontend Pages & Routing

Admin Panel (/admin/*)

Route	Component	Description
/admin/login	AdminLogin.js	Admin login form
/admin	AdminDashboard.js	Main dashboard with stats
/admin/gyms	AdminGyms.js	Gym management (CRUD)
/admin/members	AdminMembers.js	Member management
/admin/plans	AdminPlans.js	Membership plans
/admin/classes	AdminClasses.js	Class management
/admin/attendance	AdminAttendance.js	Class attendance
/admin/schedules	AdminSchedules.js	Schedule calendar
/admin/staff	AdminStaff.js	Staff management
/admin/access	AdminAccess.js	Access logs viewer
/admin/devices	AdminDevices.js	IoT device management
/admin/notifications	AdminNotifications.js	Push notifications
/admin/guests	AdminGuests.js	Guest passes
/admin/accounting	AdminAccounting.js	Financial reports
/admin/pos	AdminPOS.js	Point of Sale
/admin/analytics	AdminAnalytics.js	Advanced analytics
/admin/saas-plans	AdminSaaSPlans.js	SaaS plan config
/admin/settings	AdminSettings.js	Gym settings
/admin/templates	AdminTemplates.js	Email templates
/admin/forms	AdminForms.js	Custom forms
/admin/security	AdminSecurity.js	Security dashboard
/admin/broadcast	AdminBroadcast.js	Platform broadcasts
/admin/data	AdminData.js	Data management/export
/admin/iframes	AdminIframes.js	Embeddable widgets
/admin/gamification	AdminGamification.js	Badges config
/admin/routines	AdminRoutines.js	Workout routines
/admin/device-monitor	AdminDeviceMonitor.js	Device status

Member PWA (/app/*)

Route	Component	Description
-------	-----------	-------------

/app/login	MemberLogin.js	Member code login
/app	MemberHome.js	QR code display + quick access
/app/classes	MemberClasses.js	Book classes
/app/history	MemberHistory.js	Access history
/app/stats	MemberStats.js	Personal statistics
/app/membership	MemberMembership.js	Membership details + pay
/app/notifications	MemberNotifications.js	Notifications
/app/guests	MemberGuests.js	Invite guests
/app/achievements	MemberGamification.js	Badges & streaks
/app/routines	MemberRoutines.js	Training routines
/app/profile	MemberProfile.js	Profile settings
/app/payment-success	PaymentSuccess.js	Post-payment confirmation

Public Routes

- /register/:gymId -> PublicRegister.js (Self-registration page)
- /kiosk/:gymId -> KioskPage.js (On-premises kiosk registration)

8. Authentication & Security

JWT Authentication

Authentication uses JWT tokens with HS256 algorithm. Tokens expire after 24 hours. The token payload contains: sub (user ID), role, gym_id, and optionally impersonating/original_role fields.

```
# Token payload structure:
{
  "sub": "user-uuid",
  "role": "gym_admin",
  "gym_id": "gym-uuid",
  "exp": 1234567890
}
```

Password Security

Passwords are hashed using bcrypt with auto-generated salt. Minimum 6 characters for password changes.

IP Blocking (Brute Force Protection)

After 5 failed login attempts from the same IP within 15 minutes, the IP is blocked for 15 minutes. Applies to both admin and member logins. Super Admin can view and unblock IPs from the Security dashboard.

Impersonation

Super Admin can impersonate any Gym Admin via POST /api/auth/admin/impersonate/{gym_id}. This creates a special JWT with impersonating=true and original_role=super_admin, allowing them to see the gym admin's view without logging out.

9. QR Code System

Dynamic QR

Dynamic QR codes are regenerated every N seconds (configurable per gym: 5, 10, or 15). Each QR contains: member_id, gym_id, Unix timestamp, and an HMAC-SHA256 signature (first 16 hex chars). The data is base64url-encoded.

```
# Dynamic QR structure (before base64):
member_id|gym_id|timestamp|hmac_signature

# Static QR structure (before base64):
STATIC|member_id|gym_id|hmac_signature

# HMAC secret is set via QR_SECRET env variable
```

Static QR

For members with poor connectivity, Super Admin can assign static QR mode. Static QR codes don't expire (no timestamp check). The mode can be set per-gym or per-member.

Anti-Passback

The system implements automatic anti-passback by checking the last access log for the member. If the last log was 'entrada' (entry), the next scan becomes 'salida' (exit) and vice versa. The first scan is always treated as 'entrada'.

10. Raspberry PI Integration

Hardware Requirements

- Raspberry Pi 3B+ or newer
- 2-channel relay module (5V)
- USB QR code scanner (HID mode - acts as keyboard)
- Wiring: GPIO 12 -> Entry relay, GPIO 16 -> Exit relay
- Relays are active LOW (GPIO.HIGH = relay off)

Software Configuration

.env file on Raspberry Pi

```
# /opt/gymaccess/.env
GYMACCESS_SERVER_URL=https://c.ingresoqr.com
GYMACCESS_GYM_TOKEN=your_gym_api_token
GYMACCESS_DEVICE_ID=your_device_uuid
GYMACCESS_RELAY_ENTRADA=12
GYMACCESS_RELAY_SALIDA=16
```

Operation Flow

1. The script uses evdev to read USB QR scanners as raw input devices
2. Multiple scanners can be configured via scanner_map.json (maps device to direction)
3. When a QR is scanned, it POSTs to /api/access/validate
4. On valid response: activates the appropriate relay for 3 seconds
5. Heartbeat ping sent every 60 seconds to /api/devices/{id}/ping
6. Auto-reconnects on network errors

Running as Service

systemd service file

```
# /etc/systemd/system/gymaccess.service
[Unit]
Description=GymAccess QR Controller
After=network.target

[Service]
User=root
WorkingDirectory=/opt/gymaccess
ExecStart=/usr/bin/python3 raspberry_access_control.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

11. Payment System

Stripe Integration

Each gym can configure its own Stripe secret key. If a gym doesn't have one, the system falls back to the global STRIPE_API_KEY environment variable. Payments use Stripe Checkout Sessions for PCI compliance.

Flow: Member selects plan -> Backend creates Checkout Session -> Member pays on Stripe -> Webhook/redirect confirms -> Membership activated

MercadoPago Integration

Each gym can configure its own MercadoPago access token for Latin American markets. Uses MercadoPago Preferences API.

Manual Payments

Admins can register manual payments (cash, card, transfer) that immediately create a membership for the member.

SaaS Platform Payments

Gym owners pay the platform via Stripe subscription. Uses PLATFORM_STRIPE_KEY env variable (separate from gym-level keys).

12. SaaS Multi-Tenant Model

The platform operates as a SaaS where each gym is a tenant. Super Admin creates SaaS plans with configurable feature flags. Each gym is assigned a SaaS plan that determines what features they can access.

Feature Flags (12 toggles)

Feature Key	Description
has_qr_access	QR code access control
has_guest_passes	Guest invitation system
has_classes	Class scheduling & booking
has_pos	Point of Sale system
has_analytics	Advanced analytics dashboard
has_gamification	Badges and streak system
has_routines	Trainer-assigned workout routines
has_email_smtp	Custom SMTP email sending
has_stripe_members	Stripe payment for members
has_mercadopago	MercadoPago payment
has_iframes	Embeddable widget iframes
has_advanced_accounting	Advanced accounting reports

Gym Suspension

Super Admin can suspend gyms in two ways:

1. Manual suspension: Blocks all access (admin login, member login, API)
2. Payment suspension: Same block, triggered by unpaid SaaS subscription

13. Email System

Each gym configures its own SMTP server in Settings. The system sends emails for:

- Welcome emails (with access code)
- Membership expiration reminders (1, 3, 7 days before)
- Payment confirmations
- Custom template emails (editable per gym)

```
# Gym SMTP Configuration fields:
smtp_host:      SMTP server hostname
smtp_port:      Port (587 for STARTTLS, 465 for SSL)
smtp_user:      SMTP username
smtp_password:  SMTP password
smtp_from_email: Sender email address
```

Email Templates

Templates support variables like {gym_name}, {member_name}, {member_code}, {expiry_date}, {amount}. Each gym gets default templates on creation that can be customized.

14. Background Jobs (Cron Tasks)

Two background tasks run daily at midnight UTC via asyncio:

- Auto-Suspend: Checks all active memberships. If end_date has passed, marks membership as expired and suspends the member (if no other active membership exists).
- Expiration Reminders: Sends email reminders for memberships expiring in 1, 3, or 7 days. Only sends if the gym has SMTP configured.

Additionally, auto-suspend runs once on server startup to catch any expired memberships.

15. Deployment Guide (Plesk VPS)

Backend (c.ingresoqr.com)

```
# 1. SSH into server
ssh root@your-server

# 2. Install dependencies
apt install python3.11 python3.11-venv mongodb-org

# 3. Create app directory
mkdir -p /opt/gymaccess
cd /opt/gymaccess

# 4. Create virtual environment
python3.11 -m venv venv
source venv/bin/activate
pip install -r requirements-prod.txt

# 5. Configure .env
cp .env.example .env
# Edit MONGO_URL, JWT_SECRET, QR_SECRET, etc.

# 6. Run with uvicorn
nohup venv/bin/uvicorn server:app --host 0.0.0.0 --port 8001 &

# 7. Configure Nginx/Plesk proxy:
#   c.ingresoqr.com -> proxy_pass http://127.0.0.1:8001
```

Frontend (app.ingresoqr.com)

```
# 1. Build locally (or download from Emergent)
cd frontend
REACT_APP_BACKEND_URL=https://c.ingresoqr.com yarn build

# 2. Upload build/ folder to Plesk
#   Go to app.ingresoqr.com in Plesk File Manager
#   Upload contents of build/ to httpdocs/

# 3. Add .htaccess for SPA routing:
RewriteEngine On
RewriteBase /
RewriteRule ^index.html$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

IMPORTANT

The git clone command only syncs backend .py files. Frontend changes require rebuilding and manually uploading the build/ folder to Plesk.

16. Environment variables

Backend (.env)

Variable	Description	Default/Example
MONGO_URL	MongoDB connection string	mongodb://localhost:27017
DB_NAME	Database name	gym_access
JWT_SECRET	Secret for JWT signing	(random string)
QR_SECRET	Secret for QR HMAC signing	(random string)
STRIPE_API_KEY	Global Stripe fallback key	sk_live_...
PLATFORM_STRIPE_KEY	Stripe key for SaaS payments	sk_live_...
UPLOAD_DIR	File upload directory	/opt/gymaccess/uploads

Frontend (.env)

```
REACT_APP_BACKEND_URL=https://c.ingresoqr.com
```

17. Troubleshooting & FAQ

'Algo salio mal' error in browser

This is the ErrorBoundary catching a React crash. Check browser console (F12) for the actual error. Common causes: missing data fields, division by zero, undefined variables.

Frontend changes not appearing in production

The git clone/sync command ONLY updates backend Python files. You must rebuild the frontend (yarn build) and manually upload the build/ folder to Plesk.

QR code not validating at turnstile

Check: 1) QR_SECRET matches between backend and .env 2) Time sync between Raspberry Pi and server 3) qr_refresh_seconds + 5s tolerance 4) Use /api/access/debug-qr to diagnose

IP blocked after failed logins

IPs are blocked after 5 failed attempts in 15 minutes. Super Admin can unblock from Security dashboard, or wait 15 minutes.

Stripe payments failing

Check: 1) gym has stripe_secret_key set in Settings 2) Or STRIPE_API_KEY env variable exists 3) Key is valid (sk_live_ or sk_test_)

Member suspended automatically

The daily cron job auto-suspends members whose memberships have expired. Renew their membership to reactivate.

Raspberry Pi relay not activating

Run test_gpio.py to verify GPIO wiring. Check: 1) Running as root 2) RPi.GPIO installed 3) Correct BCM pin numbers

MongoDB connection error

Verify MONGO_URL in .env. Check MongoDB is running: systemctl status mongod